ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/324786414

A review of approximate computing techniques towards fault mitigation in HW/SW systems

Conference Paper · March 2018

DOI: 10.1109/LATW.2018.8347241

CITATIONS
28

READS
1,002

4 authors:



Alexander Aponte Moreno
National University of Colombia
13 PUBLICATIONS **86** CITATIONS
SEE PROFILE



Alejandro Moncada
National Pedagogic University, Colombia
1 PUBLICATION **28** CITATIONS
SEE PROFILE



Felipe Restrepo-Calle
National University of Colombia
84 PUBLICATIONS **818** CITATIONS
SEE PROFILE



Cesar Pedraza
National University of Colombia
57 PUBLICATIONS **361** CITATIONS
SEE PROFILE

All content following this page was uploaded by Felipe Restrepo-Calle on 15 February 2019.

The user has requested enhancement of the downloaded file.

A Review of Approximate Computing Techniques towards Fault Mitigation in HW/SW Systems

Alexander Aponte-Moreno, Alejandro Moncada, Felipe Restrepo-Calle, Cesar Pedraza
Department of Systems and Industrial Engineering
Universidad Nacional de Colombia
Bogotá D.C., Colombia
{jaapontem, amoncadap, ferestrepoca, capedrazab}@unal.edu.co

***Abstract*—Technological scaling has increased the susceptibility of logic circuits to radiation-induced transient faults, making digital devices less reliable. Although different techniques have been proposed at software, hardware, and hybrid level to mitigate these faults, all of them cause non-negligible overheads in terms of area, energy, and performance. Recently, the use of the approximate computing has taken relevance to minimize the overheads associated with the mitigation of transient faults. The approximate computing paradigm aims to increase the efficiency of a computer system at the expense of accuracy in the results. In this paper we present a review of approximate computing techniques that can be used to reduce the costs in the mitigation of radiation effects at hardware and software levels. In addition, we examine several recent works that have presented promising results in this way.**

***Keywords*—Fault Tolerance, Approximate Computing, Soft Errors.**

I. Introduction

Technological advances in the field of electronics have brought improvements such as increased integration density and computing capacity, but they have also made integrated circuits (ICs) more susceptible to external effects, like those caused by radiation [1]. This susceptibility is mainly due to the reduction of supply voltages and noise margins in modern electronic systems. The adverse effects can cause permanent damage or a temporary malfunction to a system (these are also known as *soft errors* - SE). This phenomenon has become one of the main problems to be solved in electronic systems [2] including embedded systems or microprocessor based systems.

The need to mitigate radiation-induced faults has been widely evidenced by the industry [2], [3], [4]. This fact has led to the development of fault mitigation techniques at different levels: manufacturing process level, physical layer level, circuit architecture level, and system level [2]. In this paper, we will focus on fault mitigation techniques at both circuit and system architecture levels.

From the system design perspective, the proposed works to mitigate radiation effects can be classified as pure hardware solutions, pure software approaches, and hybrid techniques [5]. These are based on some kind of redundancy: hardware, software, information, and time. Typical examples of hardware-based techniques are based on components replication at different levels, e.g., DMR (Double Modular Redundancy) for fault detection, and TMR (Triple Modular Redundancy) for fault correction [2]. Moreover, in microprocessor-based systems, it is also possible to implement fault mitigation techniques based on software, avoiding the modification or inclusion of hardware components. These can be classified into techniques for control flow protection [6], [7], [8] or data flow protection [9], [10], [11]. Furthermore, hybrid solutions combine hardware and software methods to obtain the best of both sides [12], [13], [14].

The aforementioned proposals are based on the addition of redundant components (hardware and software), which results in power, area, and performance overheads. In many cases these non-negligible costs make them unsuitable for current designs. In order to reduce redundancy's overheads, some researchers have proposed fault mitigation schemes based on partial or selective redundancy based on hardware [15], software [16], [17], or hardware/software [18], [19]. Most of these schemes focus on protecting only essential parts of the system, leaving some of them unprotected.

Recent works have proposed to use *Approximate Computing* (AC) on the design of fault tolerant systems [20], [21], [22]. The AC paradigm aims to increase the efficiency of a computer system at the expense of accuracy in the results [23]. It is important to differentiate the property of an application to tolerate inaccurate results with the ability of a system to continue operating in case of faults. The AC permits leveraging efficiency improvements on the redundancy, achieving a good trade-off between fault coverage and overheads. Thus, it is worth to explore further this promising approach in the design of fault tolerant hardware/software systems.

In this context, the objective of this paper is to present a review of AC techniques that can be used to reduce the costs in the fault mitigation at hardware and software levels. Firstly, we will present and classify the various AC techniques, including a discussion on software tools for approximate computing. Secondly, we will discuss several recent proposals that are using AC in the fault tolerant scenario. Unlike recently published reviews on AC techniques [23], [24], [25], the main contribution of this work is the review of these approaches from the fault tolerant perspective.

The rest of this paper is organized as follows. Section II presents the background on AC, and the classification of AC techniques. Next, Section III discusses the findings of the review of fault tolerance approaches using AC. Finally, Section

978-1-5386-1472-3/18/$31.00 ©2018 IEEE

IV concludes the paper and discusses future challenges in this field.

II. Approximate Computing (AC)

The AC paradigm seeks to improve performance and power efficiency taking advantage of the flexibility of applications to tolerate inaccurate results [23]. This tolerance to imprecise results has different causes such as user perceptual limitations, redundant input data, and noisy inputs [26]. It is also very dependent on the type of application. For instance, in applications such as image processing, pattern recognition and data mining, accurate results are not necessary in many cases. These are examples of imprecision-tolerant applications.

There are different techniques for the implementation of AC, which can be classified into two types: techniques at the hardware level [27], [28], [29], with less accurate and high efficient energy components; and software-level techniques [30], [31], [32], that reduce calculations or accesses to memory to improve performance at the expense of precision in the results [25].

Figure 1 illustrates different AC techniques classified according to their nature: software-based, including approaches at source code and compiler levels; or hardware-based, considering proposals at circuit and system architecture levels.

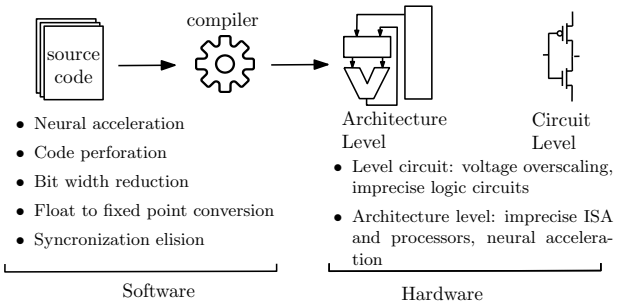


Fig. 1. Classification of different approximate computing techniques

A. Hardware techniques for AC

1) Circuit-level: At circuit level for AC, there are two main techniques: timing and functional approximate circuits.

The first one refers to the strategies based on over-scaling voltage in electronic components [33], [34], [35]. This method consists in lowering the supply voltage of the circuit to achieve a reduction of energy, but also timing errors may occur [33]. This is a fine-grain technique and only improve energy consumption [24].

The second technique is related to the design of imprecise components and the synthesis of approximated logic circuits. It is possible to improve energy efficiency in terms of area and energy principally by using inaccurate hardware. Several works have proposed approximate arithmetic circuits. For instance, approximate full-adders are presented in [36], [37], [38]; different multi-bit adders are proposed in [39], [40]; and several approximate multipliers can be found in Jiang et al. review [41].

While the design of many of these arithmetic circuits are designed manually, in general logic circuits it is better to use automated synthesis tools. In [42] a heuristic method is proposed to identify minterms that can be complemented in order to reduce the literals of a logical function from an error rate threshold. In addition, Miao et al. [43] propose a method for the approximate logical synthesis where error magnitude and frequency restrictions are taken into account [43]. First, an approximate circuit is synthesized from only magnitude error restrictions, and then, it is iteratively adjusted to achieve the error restrictions. Moreover, a method for the synthesis of approximate logical circuits called SALSA is presented in [44]. This systematic method encodes the quality constraints into logical functions and generates approximate "don't cares" by reducing the area and power of the synthesized circuit. To extend the functionality of SALSA to sequential circuits, ASLAN was presented in [45]. It is a framework that automatically synthesizes approximate versions of sequential circuits, which consume less energy.

2) Architecture: At the system architecture level, at the expense of the quality of computation (or data), AC can improve performance, energy efficiency, chip area in the case of processors or density for memories [46], and storage [47].

For processor architectures many approaches have been proposed to run energy-efficient code segments or single instructions. For instance, in [48] a Scalable Effort Recognition and Mining Processor is presented. This consists of several Processing Elements (PEs), each with a Data Path where it is possible to reduce the precision of the variables that are processed on it. Furthermore, Sampson et al. propose a hardware model based on approximation-aware ISA (Instruction Set Architecture) extensions to approximate storage (registers, cache and main memory) and operations (approximate floating point operations) [49]. In the same way, a microarchitecture design called Truffle is presented in [50]. It supports an ISA extension to approximate data and operations, and it is based on dual-voltage operation. Another proposed processor design based on ISA extensions is presented in [51]. The proposal is called QUORA and consists of a quality programmable processor based on precision scaling.

Alternatively, performance can be improved by transforming segments of code into a neural-inspired algorithm running on hardware accelerators. A proposal to accelerate code based on Parrot transformation is shown in [27]. In this work, a neural network is trained to replace a portion of code that the programmer determines is susceptible to acceleration. In the compilation stage, the configuration is generated for an accelerator called a neural processing unit (NPU) that implements the trained neural network. As NPU, a digital ASIC was used.

B. Software techniques and tools for AC

In this section, the main techniques for AC at the software level are shown. Next, the most important tools to implement these techniques are presented.

1) Software techniques: the most important techniques for AC based on software are: *code perforation*, *bit width reduc-*

tion, *float point to fixed point conversion*, and *synchronization elision*.

Code perforation: this technique is based on identifying parts of code that can be discarded without exceeding an error threshold [52]. This technique includes loop perforation [53], [54], which consists of reducing the number of iterations in a loop in order to reduce computational costs.

Bit width reduction: by reducing the precision (number of bits) of a floating point number, it is possible to improve performance and power consumption. Based on this principle, an analysis tool called PRECIMONIUS is presented in [55]. This tool helps to automate the process of fine tuning the precision of floating point programs. In addition, a method to hide information in IoT (Internet of Things) devices, which is based on the AC paradigm, is presented in [56]. It consists in using some less significant bits of the number mantissa in IEEE-754 format of simple precision for such purpose.

Float point to fixed point conversion: a floating point program can be approximated to a fixed point program to reduce energy costs [57]. In [32], a tool-chain is presented to transform floating point signal processing programs into their fixed point equivalents. An algorithm looks for the minimum precision in the mantissa in a floating point code, taking into account restrictions with respect to the output error.

Synchronization elision: synchronization overheads are one of the main problems of parallel applications for multicores. A good opportunity to improve performance and efficiency is to relax the synchronization [58]. Misailovic et al. present a system that generates a space of alternatives of parallel programs relaxing the synchronization primitives [59]. The system is called Dupstep and it also explores this design space to find the most efficient versions of the program.

Table I summaries the most relevant AC techniques that have been discussed up to this point.

2) Software tools for AC: Table II presents the classification of software tools for approximate computing.

Software techniques for AC can be applied on some stages such as: design, preprocessing, compilation, and execution. They can be called from annotations in the source code, or transformation based rules applied to source code and object code.

TABLE I
SUMMARY OF APPROXIMATE COMPUTING TECHNIQUES

AC Technique	*Operation Level*	*Approximation Level*	*Related Works*
Voltage Over-scaling	Circuit	Fine-grained	[33], [34], [35]
Imprecise logic circuits	Circuit	Fine-grained	[36], [37], [38], [39], [40], [42], [43], [44], [45]
Imprecise ISA and Processors	Architecture	Fine-grained	[48], [49], [50], [51]
Neural acceleration	Architecture/ Software	Coarse-grained	[27]
Code perforation	Software	Coarse-grained	[52], [53], [54]
Bit-width Reduction	Software	Fine-grained	[55], [56]
Float to fixed point conversion	Software	Fine-grained	[32], [57]
Synchronization Elision	Software	Coarse-grained	[58], [59]

In AC the source code of a program can be classified into two main categories [49]: *error-sensitive* (ES): the code is critical and cannot be approximated; and *error-resilient* (ER): the code is not critical and can be approximated, this means that AC techniques can be applied. There are multiple tools that allow the developer to identify the code category by means of annotations in the source code or provide tools for automatic analysis [60].

When a loop in the source code belongs to the *error-resilient* category, the *loop perforation* technique can be applied, which seeks to obtain an approximate result by reducing the number of times that an invariable cycle is executed. In other cases, it is possible to relax the value of the variables (approximate) in order to minimize the mantissa length, and thus, simplify floating point operations.

AC, as an imprecise computation paradigm, uses other imprecise computation methods to take advantage of, and its main difference could be that it is not necessary to change the flow of a program. This allows AC to be applied in a larger number of applications. In AC it is possible to implement other computational concepts such as *soft computing*. This implies that inaccurate solutions are found for difficult problems (of type *non-polynomial* NP) in polynomial or constant times [60]. Some examples of these include the application domains for *neural networks* (NN), *machine learning algorithms* (MA), *fuzzy logic* (FL), *genetic algorithms* (GA), and meta-heuristic intelligence [60].

III. Approximate Computing Techniques for Fault Tolerance

AC techniques are increasingly taken into account to reduce energy consumption, improve performance and reduce chip area for different applications. Recently, this tendency has been extended to the field of fault tolerant systems design. This has permitted to reduce overheads associated to redundancy. In this Section, we review the most relevant works that involve AC techniques in the efforts of fault mitigation.

In [65], an algorithm is proposed for the synthesis of approximate logical circuits, based on converting minterms into "Don't Cares". The algorithm is iterative and it is aimed at reducing Boolean expressions. A non-intrusive solution for concurrent error detection (CED) based on the approximate circuits is described. It presents fine grained trade-offs between overheads and coverage of the CED. The solution was simulated. For an 81% coverage, it achieves an area and energy overhead of 25% and 34%, respectively. The approximate circuit delay is on average 38% less than the original.

An architecture called ERSA is presented in [66]. This consists of a mixture of cores. Some reliable cores and a large number of non-reliable cores, in order to reduce the overheads of traditional redundancy techniques.

In [67], an HW/SW architecture with a declarative resilience scheme is presented. The proposal is based on the NMR technique for multi-core architecture. In this case, the parts of code that have a great impact on the results of the program

TABLE II
SOFTWARE TOOLS FOR APPROXIMATE COMPUTING

Relevant Software tools in AC	Software features									
	Code perforation	Bit width reduction	Float point to fixed point conversion	Synchronization elision	Code availibility via repository	Licence	Related documents	Soft computing techniques	Source code last update	Main features
Enerj	✓		✓		✓	OS	[49]		Sep. 2013	Isolate ER and ES by annotations
Accept	✓	✓	✓	✓	✓	MIT	[61]	NN	Apr. 2017	Automated analysis and optimizations
ApproxANN		✓	✓			-	[62]	NN ML	-	Neural Networks, gradient descend
Approxit	✓		✓			-	[63]	ML	-	Optimize by incremental and adaptative strategy
Approxilyzer		✓	✓		✓	UI/NSCA OS	[64]		Jan. 2017	Heuristics, dynamic and static analysis
iACT	✓	✓	✓		✓	OS	[60]		Mar. 2014	Measures the impact of AC implementation

are identified and classified as crucial. A Hardware Redundant Execution (HaRE) scheme is activated for the crucial code and deactivated for the non-crucial one. Errors in the non-crucial code only affect the accuracy of the program. With this proposal a total error coverage is achieved and a reduction of the overhead is achieved from 1.75 to 1.38× compared to systems with no resilience support.

Gomez et al. [68] propose a technique based on approximate logical circuits called Approximate-TMR (ATMR). Permutations of inputs and topology at the transistor level are used to improve fault masking. It was shown, by means of simulations and fault injection, that 99% of fault coverage can be obtained with an area overhead of 150% instead of 200% of traditional TMR. This work is extended in [69] where is proposed an ATMR scheme, called Full-Approximated TMR (Full-ATMR), with only approximate modules. By means of fault injection campaigns, it was shown that for a fault coverage of more than 95% the area overhead can be reduced to 120%, whereas a 75% of fault coverage can be achieved with 0% overhead.

Salehi et al. [70] propose an efficient system for managing reliability in multi-core systems called DRVS. This is based on the use of dynamic redundancy in conjunction with voltage-frequency scaling according to the application. The authors show 28% - 60% in power reduction with this system.

Moreover, in [22] a systematic methodology to develop approximate circuits for TMR is presented. An effective use of the error threshold is made, to obtain minimum over costs compared to previous works. This method consists of approximate logical synthesis based on minterms complements. Expansion and reduction of prime implicants are used.

Sanchez et al. [21] present a proposal to implement partial TMR in FPGA using approximate logic circuits. The line substitution technique was used to generate approximate circuits, reducing the number of LUTs or their size. This technique consists of replacing some lines of the circuit with logical constants. Probabilistic methods were used to identify the most critical lines of the circuit.

A framework for A.C. called *Approxilyzer* is presented in [64]. This tool helps in the design process by analyzing the effects of errors at the instruction level and identifying possible instructions for approximation. The authors show that the tool can be used to fine tune the output quality vs. resilience vs. overheads.

More recently, authors in [71] present two proposals for mitigation of errors using inexact (approximate) computing. The first one is a technique called Inexact Double Modular Redundancy (IDMR), base on DMR. This consists on comparing the outputs of two modules and establishing the difference between them. If this difference is larger than a threshold, a voter generates a error signal. Otherwise, the output will be the average of the modules outputs. The second proposal integrates TMR with the IDMR module of the previous proposal. In this case, the outputs of three modules are compared, but unlike traditional TMR, valid results will be considered if the difference is below a threshold.

At the software level, a proposal is presented to reduce cost overruns in NMR schemes for multi-core processors [72]. The NMR system consists of a combination of exact and approximate versions of a task to be executed in different cores. The advantage is that the approximate versions have shorter execution times compared to the exact ones, improving performance and energy efficiency. In addition, several of them can be executed in a core, reducing area overheads. The authors also present a tool called *LEXACT*, which helps in the design process by finding code sections that can be approximated (by loop perforation or precision scaling). This tool also checks that the output satisfies the threshold according to predefined metrics, and adjusts or not the approximation level.

Table III summarizes the proposals presented in this Section, i.e., where AC has been used for the reduction of overheads associated to fault tolerance.

IV. CONCLUSIONS AND FUTURE CHALLENGES

Efforts to mitigate radiation-induced faults entail energy, area and performance overheads. Several works to minimize the costs associated to fault tolerance at different levels have been proposed. The use of the AC paradigm has shown great opportunities to improve the efficiency of a system, at the cost of a reduction in the accuracy of results. These opportunities

TABLE III
PROPOSALS USING APPROXIMATE COMPUTING FOR FAULT TOLERANCE

Proposal	*AC Technique*	*FT Strategy*
Choudhury et al., 2008 [65]	Imprecise logic circuits	Concurrent error detection
Cho et, al., 2012 [66]	Imprecise logic circuits	NMR
Shi et al., 2015 [67]	Other	NMR
Gomez et al., 2013 - 2015 [68], [69]	Imprecise logic circuits	TMR
Salehi et al., 2015 [70]	Voltage scaling	NMR
Arifeen et al., 2016 [22]	Imprecise logic circuits	TMR
Sanchez et al., 2016 [21]	Imprecise logic circuits	TMR
Chen et al., 2017 [71]	Other	NMR
Baharvand et al., 2017 [72]	Loop perforation Precision scaling	NMR

have recently been used to try to reduce the overheads of different fault tolerance schemes, showing promising results.

In this paper we have made a review of the main techniques for AC at the software and hardware level, as well as the works that make use of them to minimize the overheads associated to fault tolerance. It should be noted that most of these works are at circuit level, and they constitute specific solutions. Hence, there is still a long way to go in the use of AC in the design of fault tolerant systems. As future work, we will explore more systematic ways to reduce overheads in fault tolerant systems using AC.

REFERENCES

[1] P. Shivakumar, M. D. Kistler, S. W. Keckler, D. C. Burger, and L. Alvisi, "Modeling the effect of technology trends on the soft error rate of combinational logic," *Dependable Systems and Networks, 2002. DSN 2002. Proceedings. International Conference on*, pp. 389–398, 2002.

[2] ECSS, *Techniques for radiation effects mitigation in ASICs and FPGAs handbook (1 September 2016) — European Cooperation for Space Standardization*. ESA Requirements and Standards Division, 2016.

[3] R. Fulton and R. Vandermolen, *Airborne electronic hardware design assurance : a practitioner's guide to RTCA/DO-254*. CRC Press, 2014.

[4] IEC, "INTERNATIONAL STANDARD Process management for avionics Atmospheric radiation effects Part 1: Accommodation of atmospheric radiation effects via single event effects within avionics electronic equipment," 2012.

[5] A. Martínez-Álvarez, S. Cuenca-Asensi, and F. Restrepo-Calle, "Soft Error Mitigation in Soft-Core Processors," in *FPGAs and Parallel Architectures for Aerospace Applications*, F. Kastensmidt and P. Rech, Eds. Springer International Publishing, 2016, ch. 16, pp. 239–258.

[6] A. Benso, S. Di Carlo, G. Di Natale, P. Prinetto, and L. Tagliaferri, "Control-flow checking via regular expressions," in *Proceedings 10th Asian Test Symposium*. IEEE, 2001, pp. 299–303.

[7] Rajesh Venkatasubramanian, J. Hayes, and B. Murray, "Low-cost on-line fault detection using control flow assertions," in *9th IEEE On-Line Testing Symposium, 2003. IOLTS 2003.* IEEE Comput. Soc, 2003, pp. 137–143.

[8] O. Goloubeva, M. Rebaudengo, M. Sonza Reorda, and M. Violante, "Soft-error detection using control flow assertions," in *Proceedings. 16th IEEE Symposium on Computer Arithmetic*. IEEE Comput. Soc, 2003, pp. 581–588.

[9] A. Avizienis, "The N-Version Approach to Fault-Tolerant Software," *IEEE Transactions on Software Engineering*, vol. SE-11, no. 12, pp. 1491–1501, dec 1985.

[10] N. Oh, S. Mitra, and E. McCluskey, "ED/sup 4/I: error detection by diverse data and duplicated instructions," *IEEE Transactions on Computers*, vol. 51, no. 2, pp. 180–199, 2002.

[11] N. Oh and E. J. McCluskey, "Error detection by selective procedure call duplication for low energy consumption," *IEEE Transactions on Reliability*, vol. 51, no. 4, pp. 392–402, 2002.

[12] P. Bernardi, L. Bolzani Poehls, M. Grosso, and M. Sonza Reorda, "A Hybrid Approach for Detection and Correction of Transient Faults in SoCs," *IEEE Transactions on Dependable and Secure Computing*, vol. 7, no. 4, pp. 439–445, oct 2010.

[13] J. R. Azambuja, Â. Lapolli, L. Rosa, and F. L. Kastensmidt, "Detecting SEEs in microprocessors through a non-intrusive hybrid technique," *IEEE Transactions on Nuclear Science*, vol. 58, no. 3 PART 2, pp. 993–1000, 2011.

[14] R. G. Ragel and S. Parameswaran, "A hybrid hardware–software technique to improve reliability in embedded processors," *ACM Transactions on Embedded Computing Systems*, vol. 10, no. 3, pp. 1–16, 2011.

[15] V. K. Reddy, E. Rotenberg, and S. Parthasarathy, "Understanding prediction-based partial redundant threading for low-overhead, high-coverage fault tolerance," *ACM SIGARCH Computer Architecture News*, vol. 34, no. 5, p. 83, 2006.

[16] E. Chielle, J. R. Azambuja, R. S. Barth, F. Almeida, and F. L. Kastensmidt, "Evaluating selective redundancy in data-flow software-based techniques," *IEEE Transactions on Nuclear Science*, vol. 60, no. 4, pp. 2768–2775, 2013.

[17] F. Restrepo-Calle, A. Martínez-Álvarez, S. Cuenca-Asensi, and A. Jimeno-Morenilla, "Selective SWIFT-R. A Flexible Software-Based Technique for Soft Error Mitigation in Low-Cost Embedded Systems," *Journal of Electronic Testing*, vol. 29, no. 6, pp. 825–838, dec 2013.

[18] X. Vera, J. Abella, J. Carretero, and A. González, "Selective replication: a lightweight technique for soft errors," *ACM Transactions on Computer Systems*, vol. 27, no. 4, pp. 1–30, 2009.

[19] A. Martínez-Álvarez, F. Restrepo-Calle, S. Cuenca-Asensi, L. M. Reyneri, A. Lindoso, and L. Entrena, "A hardware-software approach for on-line soft error mitigation in interrupt-driven applications," *IEEE Trans Dependable and Secure Comp.*, vol. 13, no. 4, pp. 502–508, Jul 2016.

[20] I. A. C. Gomes, M. G. A. Martins, A. I. Reis, and F. L. Kastensmidt, "Exploring the use of approximate TMR to mask transient faults in logic with low area overhead," *Microelectronics Reliability*, vol. 55, no. 9-10, pp. 2072–2076, 2015.

[21] A. J. Sanchez-Clemente, L. Entrena, and M. Garcia-Valderas, "Partial TMR in FPGAs Using Approximate Logic Circuits," *IEEE Transactions on Nuclear Science*, vol. 63, no. 4, pp. 2233–2240, aug 2016.

[22] T. Arifeen, A. S. Hassan, H. Moradian, and J. A. Lee, "Probing Approximate TMR in Error Resilient Applications for Better Design Tradeoffs," in *Proceedings - 19th Euromicro Conference on Digital System Design, DSD 2016*, 2016, pp. 637–640.

[23] S. Mittal, "A Survey of Techniques for Approximate Computing," *ACM Computing Surveys*, vol. 48, no. 4, pp. 1–33, mar 2016.

[24] T. Moreau, J. S. Miguel, M. Wyse, J. Bornholt, A. Alaghi, L. Ceze, N. E. Jerger, and A. Sampson, "A Taxonomy of General Purpose Approximate Computing Techniques," *IEEE Embedded Systems Letters*, no. 99, pp. 1–1, 2017.

[25] Q. Xu, T. Mytkowicz, and N. S. Kim, "Approximate Computing: A Survey," *IEEE Design and Test*, vol. 33, no. 1, pp. 8–22, feb 2016.

[26] J. Han and M. Orshansky, "Approximate computing: An emerging paradigm for energy-efficient design," *Proceedings - 2013 18th IEEE European Test Symposium, ETS 2013*, 2013.

[27] H. Esmaeilzadeh, A. Sampson, L. Ceze, and D. Burger, "Neural Acceleration for General-Purpose Approximate Programs," in *Microarchitecture (MICRO), 2012 45th Annual IEEE/ACM International Symposium*, vol. 33, no. 3. IEEE, 2012.

[28] A. Alaghi and J. P. Hayes, "STRAUSS: Spectral Transform Use in Stochastic Circuit Synthesis," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 34, no. 11, pp. 1770–1783, nov 2015.

[29] M. Van Leussen, J. Huisken, L. Wang, H. Jiao, and J. P. de Gyvez, "Reconfigurable Support Vector Machine Classifier with Approximate Computing," in *2017 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. IEEE, jul 2017, pp. 13–18.

[30] X. He, G. Yan, Y. Han, and X. Li, "ACR: Enabling computation reuse for approximate computing," *Proceedings of the Asia and South Pacific Design Automation Conference, ASP-DAC*, vol. 25-28-Janu, pp. 643–648, jan 2016.

[31] L. McAfee and K. Olukotun, "EMEURO: A framework for generating multi-purpose accelerators via deep learning," in *2015 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, feb 2015, pp. 125–135.

[32] N.-M. Ho, E. Manogaran, W.-F. Wong, and A. Anoosheh, "Efficient floating point precision tuning for approximate computing," in *2017 22nd Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, jan 2017, pp. 63–68.

[33] D. Mohapatra, V. K. Chippa, A. Raghunathan, and K. Roy, "Design of voltage-scalable meta-functions for approximate computing," in *2011 Design, Automation & Test in Europe*. IEEE, mar 2011, pp. 1–6.

[34] R. Hegde and N. Shanbhag, "Soft digital signal processing," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 9, no. 6, pp. 813–823, dec 2001.

[35] V. K. Chippa, D. Mohapatra, K. Roy, S. T. Chakradhar, and A. Raghunathan, "Scalable Effort Hardware Design," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 22, no. 9, pp. 2004–2016, sep 2014.

[36] V. Gupta, D. Mohapatra, A. Raghunathan, and K. Roy, "Low-Power Digital Signal Processing Using Approximate Adders," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 32, no. 1, pp. 124–137, jan 2013.

[37] Z. Yang, A. Jain, J. Liang, J. Han, and F. Lombardi, "Approximate XOR/XNOR-based adders for inexact computing," in *2013 13th IEEE International Conference on Nanotechnology (IEEE-NANO 2013)*. IEEE, aug 2013, pp. 690–693.

[38] H. R. Mahdiani, A. Ahmadi, S. M. Fakhraie, and C. Lucas, "Bio-Inspired Imprecise Computational Blocks for Efficient VLSI Implementation of Soft-Computing Applications," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 57, no. 4, pp. 850–862, apr 2010.

[39] Ning Zhu, W. L. Goh, and K. S. Yeo, "An enhanced low-power high-speed Adder For Error-Tolerant application," in *Proceedings of the 2009 12th International Symposium on Integrated Circuits*, 2009.

[40] N. Zhu, W. L. Goh, G. Wang, and K. S. Yeo, "Enhanced low-power high-speed adder for error-tolerant application," in *2010 International SoC Design Conference*. IEEE, nov 2010, pp. 323–327.

[41] C. L. Honglan Jiang and Naman Maheshwari, "A comparative evaluation of approximate multipliers," in *Nanoscale Architectures (NANOARCH), 2016 IEEE/ACM International Symposium on*, 2016.

[42] Doochul Shin and S. K. Gupta, "A new circuit simplification method for error tolerant applications," in *2011 Design, Automation & Test in Europe*. IEEE, mar 2011, pp. 1–6.

[43] J. Miao, A. Gerstlauer, and M. Orshansky, "Approximate logic synthesis under general error magnitude and frequency constraints," in *2013 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, nov 2013, pp. 779–786.

[44] S. Venkataramani, A. Sabne, V. Kozhikkottu, K. Roy, and A. Raghunathan, "SALSA: Systematic logic synthesis of approximate circuits," in *Proceedings of the 49th Annual Design Automation Conference on - DAC '12*. New York, New York, USA: ACM Press, 2012, p. 796.

[45] A. Ranjan, A. Raha, S. Venkataramani, K. Roy, and A. Raghunathan, "ASLAN: Synthesis of approximate sequential circuits," in *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2014*. New Jersey: IEEE Conference Publications, 2014, pp. 1–6.

[46] I. J. Chang, D. Mohapatra, and K. Roy, "A Priority-Based 6T/8T Hybrid SRAM Architecture for Aggressive Voltage Scaling in Video Applications," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 21, no. 2, pp. 101–112, feb 2011.

[47] A. Sampson, J. Nelson, K. Strauss, and L. Ceze, "Approximate storage in solid-state memories," *2013 46th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 25–36, 2013.

[48] V. K. Chippa, S. Venkataramani, S. T. Chakradhar, K. Roy, and A. Raghunathan, "Approximate computing: An integrated hardware approach," in *2013 Asilomar Conference on Signals, Systems and Computers*. IEEE, nov 2013, pp. 111–117.

[49] A. Sampson, W. Dietl, E. Fortuna, D. Gnanapragasam, L. Ceze, and D. Grossman, "EnerJ: approximate data types for safe and general low-power computation," in *Proc. 32nd ACM SIGPLAN PLDI '11*. New York, New York, USA: ACM Press, 2011, p. 164.

[50] H. Esmaeilzadeh, A. Sampson, L. Ceze, D. Burger, H. Esmaeilzadeh, A. Sampson, L. Ceze, D. Burger, H. Esmaeilzadeh, A. Sampson, L. Ceze, and D. Burger, "Architecture support for disciplined approximate programming," in *Proc. 17th ASPLOS '12*, vol. 47, no. 4. New York, New York, USA: ACM Press, 2012, p. 301.

[51] S. Venkataramani, V. K. Chippa, S. T. Chakradhar, K. Roy, and A. Raghunathan, "Quality programmable vector processors for approximate computing," in *2013 46th Annual IEEE/ACM MICRO*, 2013, pp. 1–12.

[52] A. Agarwal, M. Rinard, S. Sidiroglou, S. Misailovic, and H. Hoffmann, "Using Code Perforation to Improve Performance, Reduce Energy Consumption, and Respond to Failures," MIT, Tech. Rep., 2009.

[53] S. Misailovic, S. Sidiroglou, H. Hoffmann, and M. Rinard, "Quality of service profiling," in *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering - ICSE '10*, vol. 1. New York, New York, USA: ACM Press, 2010, p. 25.

[54] S. Sidiroglou-Douskos, S. Misailovic, H. Hoffmann, and M. Rinard, "Managing performance vs. accuracy trade-offs with loop perforation," in *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering - SIGSOFT/FSE '11*. New York, New York, USA: ACM Press, 2011, p. 124.

[55] C. Rubio-González, C. Nguyen, H. D. Nguyen, J. Demmel, W. Kahan, K. Sen, D. H. Bailey, C. Iancu, and D. Hough, "Precimonious," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis on - SC '13*. New York, New York, USA: ACM Press, 2013, pp. 1–12.

[56] M. Gao and G. Qu, "A novel approximate computing based security primitive for the Internet of Things," in *2017 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, may 2017, pp. 1–4.

[57] T. M. Aamodt and P. Chow, "Compile-time and instruction-set methods for improving floating- to fixed-point conversion accuracy," *ACM Transactions on Embedded Computing Systems*, vol. 7, no. 3, pp. 1–27, apr 2008.

[58] L. Renganarayana, V. Srinivasan, R. Nair, and D. Prener, "Programming with relaxed synchronization," in *Proceedings of the 2012 ACM workshop on Relaxing synchronization for multicore and manycore scalability - RACES '12*. New York, New York, USA: ACM Press, 2012, p. 41.

[59] S. Misailovic, S. Sidiroglou, and M. C. Rinard, "Dancing with uncertainty," in *Proceedings of the 2012 ACM workshop on Relaxing synchronization for multicore and manycore scalability - RACES '12*. New York, New York, USA: ACM Press, 2012, p. 51.

[60] A. K. Mishra, R. Barik, and S. Paul, "iACT: A Software-Hardware Framework for Understanding the Scope of Approximate Computing."

[61] A. Sampson, A. Baixo, B. Ransford, T. Moreau, J. Yip, L. Ceze, and M. Oskin, "ACCEPT: A Programmer-Guided Compiler Framework for Practical Approximate Computing."

[62] Q. Zhang, T. Wang, Y. Tian, F. Yuan, and Q. Xu, "ApproxANN: An Approximate Computing Framework for Artificial Neural Network," in *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2015*. New Jersey: IEEE Conference Publications, 2015, pp. 701–706.

[63] Qian Zhang, F. Yuan, R. Ye, and Q. Xu, "ApproxIt: An approximate computing framework for iterative methods," in *2014 51st ACM/EDAC/IEEE Design Automation Conference (DAC)*. IEEE, jun 2014, pp. 1–6.

[64] R. Venkatagiri, A. Mahmoud, S. K. S. Hari, and S. V. Adve, "Approxilyzer: Towards a systematic framework for instruction-level approximate computing and its application to hardware resiliency," in *2016 49th Annual IEEE/ACM MICRO*. IEEE, oct 2016, pp. 1–14.

[65] M. R. Choudhury and K. Mohanram, "Approximate logic circuits for low overhead, non-intrusive concurrent error detection," in *2008 DATE*, vol. 1, no. 0. IEEE, mar 2008, pp. 903–908.

[66] H. Cho, L. Leem, and S. Mitra, "ERSA: Error Resilient System Architecture for Probabilistic Applications," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 31, no. 4, pp. 546–558, apr 2012.

[67] Q. Shi, H. Hoffmann, and O. Khan, "A Cross-Layer Multicore Architecture to Tradeoff Program Accuracy and Resilience Overheads," *IEEE Computer Architecture Letters*, vol. 14, no. 2, pp. 85–89, jul 2015.

[68] I. A. C. Gomes and F. G. L. Kastensmidt, "Reducing TMR overhead by combining approximate circuit, transistor topology and input permutation approaches," *Chip in Curitiba 2013 - SBCCI 2013: 26th Symposium on Integrated Circuits and Systems Design*, 2013.

[69] I. a. C. Gomes, M. Martins, A. Reis, and F. L. Kastensmidt, "Using only redundant modules with approximate logic to reduce drastically area overhead in TMR," in *2015 16th Latin-American Test Symposium (LATS)*. IEEE, mar 2015, pp. 1–6.

[70] M. Salehi, M. K. Tavana, S. Rehman, F. Kriebel, M. Shafique, A. Ejlali, and J. Henkel, "DRVS: Power-efficient reliability management through Dynamic Redundancy and Voltage Scaling under variations," in *2015 IEEE/ACM ISLPED*. IEEE, jul 2015, pp. 225–230.

[71] K. Chen, J. Han, and F. Lombardi, "Two Approximate Voting Schemes for Reliable Computing," *IEEE Transactions on Computers*, vol. 66, no. 7, pp. 1227–1239, jul 2017.

[72] F. Baharvand and S. Ghassem Miremadi, "LEXACT: Low Energy N-Modular Redundancy Using Approximate Computing for Real-Time Multicore Processors," *IEEE Transactions on Emerging Topics in Computing*, pp. 1–1, 2017.

View publication stats